

ASSIGNMENT No :3

TITLE: Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Theory

Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Conclusion

This Java application successfully demonstrates the core Object-Oriented Programming principles of method overloading and static class design. By implementing overloaded addition methods for different parameter types and counts, the application achieves compile-time polymorphism, providing a flexible and unified interface for performing calculations on both integers and decimals. Furthermore, incorporating a static counter variable and a static getter method illustrates how class-level members can efficiently track shared state across invocations without duplicating memory for individual instances. Overall, combining method overloading with static design elements produces a clean, highly reusable, and memory-efficient program structure.

Exercise

1. Develop a **Calculator program** using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.

Code:

```
1  class Calculator {
2      private static int calculationCount = 0;
3
4      public int add(int a, int b) {
5          calculationCount++;
6          return a + b;
7      }
8
9      public double add(double a, double b) {
10         calculationCount++;
11         return a + b;
12     }
13     public int add(int a, int b, int c) {
14         calculationCount++;
15         return a + b + c;
16     }
17     public static int getCalculationCount() {
18         return calculationCount;
19     }
20
21     Run main | Debug main
22     public static void main(String[] args) {
23         Calculator calc = new Calculator();
24
25         System.out.println("Sum (int): " + calc.add(5, 10));
26         System.out.println("Sum (double): " + calc.add(5.5, 10.5));
27         System.out.println("Sum (3 ints): " + calc.add(5, 10, 15));
28         System.out.println("Total calculations made: " + Calculator.getCalculationCount());
29     }
```

Output:

```
Listening on 51948
User program running
Sum (int): 15
Sum (double): 16.0
Sum (3 ints): 30
Total calculations made: 3
User program finished
```

2. Develop a **Restaurant Billing Application** where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.

Code:

```
1  class RestaurantBilling {
2      private static int totalOrders = 0;
3      private static double totalRevenue = 0.0;
4      private static final double TAX_RATE = 0.05;
5      private static final double PACKAGING_FEE = 15.0;
6      public double calculateBill(double itemTotal) {
7          double tax = itemTotal * TAX_RATE;
8          double finalAmount = itemTotal + tax;
9          recordOrder(finalAmount);
10         return finalAmount;
11     }
12     public double calculateBill(double itemTotal, boolean isTakeaway) {
13         double tax = itemTotal * TAX_RATE;
14         double packaging = isTakeaway ? PACKAGING_FEE : 0.0;
15         double finalAmount = itemTotal + tax + packaging;
16
17         recordOrder(finalAmount);
18         return finalAmount;
19     }
20     public double calculateBill(double itemTotal, double distanceInKm) {
21         double tax = itemTotal * TAX_RATE;
22         double deliveryCharge = distanceInKm * 10.0; // ₹10 per km
23         double finalAmount = itemTotal + tax + deliveryCharge;
24
25         recordOrder(finalAmount);
26         return finalAmount;
27     }
28
29     private static void recordOrder(double orderAmount) {
30         totalOrders++;
31         totalRevenue += orderAmount;
32     }
33     public static int getTotalOrders() {
34         return totalOrders;
35     }
36     public static double getTotalRevenue() {
37         return totalRevenue;
38     }
39
40     Run main | Debug main
41     public static void main(String[] args) {
42         RestaurantBilling billingSystem = new RestaurantBilling();
43         double bill1 = billingSystem.calculateBill(500.0);
44         System.out.println("Dine-In Bill: ₹" + bill1);
45         double bill2 = billingSystem.calculateBill(350.0, true);
46         System.out.println("Takeaway Bill: ₹" + bill2);
47         double bill3 = billingSystem.calculateBill(800.0, 4.5);
48         System.out.println("Delivery Bill: ₹" + bill3);
49         System.out.println("SUMMARY REPORT:");
50         System.out.println("Total Orders Processed: " + RestaurantBilling.getTotalOrders());
51         System.out.println("Total Revenue Generated: ₹" + RestaurantBilling.getTotalRevenue());
52     }
53 }
```

Output:

```
Listening on 64242
User program running
User program finished
Dine-In Bill: ₹525.0
Takeaway Bill: ₹382.5
Delivery Bill: ₹885.0
SUMMARY REPORT:
Total Orders Processed: 3
Total Revenue Generated: ₹1792.5
```